UNIVERSIDAD SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DE OCCIDENTE

DIVISIÓN DE CIENCIAS DE LA INGENIERÍA

CARRERA DE CIENCIAS Y SISTEMAS

SEGUNDO SEMESTRE 2026

TEORÍA DE SISTEMAS 1

ING. IVÁN ESTEBAN

CLAUDA LOVERS

(SISTEMA DE CONTROL DE GIMNASIOS)

Quetzaltenango, 29 de Octubre de 2026

Contenido

Contenido **2**
1. Stack tecnológico y arquitectura **3**
1.1 Tecnologías y versiones 3
1.2 Patrón arquitectónico 4
1.3 Flujo general de una petición 5
2. Entorno local y setup (Developer Onboarding) **5**
2.1 Requisitos previos 5
2.2 Variables de entorno (.env) 6
2.3 Guía de levantamiento local 7
Opción A: con Docker Compose (recomendada) 7
Opción B: sin Docker (ejecución nativa con Maven) 8
2.4 Comandos clave 8
3. Base de datos y persistencia **8**
3.1 Diagrama Entidad-Relación (DER) 8
3.2 Estrategia de migraciones 13
Ejecutar las migraciones existentes 13
Crear una nueva migración 14
3.3 Datos de prueba (seeds) 14
4. Estructura del proyecto **15**
4.1 Responsabilidad de cada carpeta 15
5. Seguridad y autenticación **16**
5.1 Flujo de autenticación 16
5.2 Gestión de sesión 16
5.3 Matriz de permisos (RBAC) 17
5.4 Políticas de CORS y Rate Limiting 17
CORS 17
Rate Limiting 18
5.5 Buenas prácticas de seguridad 18
6. Swagger, excepciones y tareas en segundo plano **18**
6.1 Swagger UI 18
6.2 Formato estándar de respuesta de error 19
6.3 Cron jobs y colas de trabajo 19
7. Estrategia de tests y CI/CD **20**
7.1 Pruebas 20
7.2 Pipeline de despliegue 20
Integración continua (CI) 20
Entrega continua (CD) 21

Secretos del pipeline..... 21
Reversión..... 22
8. Solución de problemas frecuentes..... 22
Anexo A. Archivo .env.example..... 23
Anexo B. Glosario..... 23

1. Stack tecnológico y arquitectura

1.1 Tecnologías y versiones

Componente	Tecnología	Versión / Detalle	Propósito
Lenguaje y runtime	Java (JDK)	21 (LTS)	Lenguaje de la aplicación y máquina virtual de ejecución
Framework principal	Spring Boot	4.x (Spring Web MVC, Security 6, Data JPA, Validation)	API REST, seguridad, persistencia y validación
Base de datos principal	MariaDB	11	Persistencia relacional
ORM	Spring Data JPA	Hibernate como implementación JPA	Mapeo objeto-relacional y repositorios
Migraciones	Flyway	Versionadas V1 a V8	Control de versiones del esquema, ejecución automática
Seguridad	Spring Security 6 + JJWT	—	Autenticación con JWT y control de acceso por roles
Tareas en segundo plano	Spring Schedulers	Cron + despachador de cola	Recordatorios de vencimiento; no hay broker externo
Correo	Resend API	—	Notificaciones por correo electrónico
Documentación API	SpringDoc OpenAPI 3 / Swagger UI	—	Especificación y portal interactivo
Build	Maven	3.9+ (wrapper ./mvnw)	Compilación, pruebas y empaquetado
Contenedores	Docker y Docker Compose	Compose v2 (20+)	Empaquetado y orquestación
Proxy inverso	Nginx	—	Entrada HTTP en producción
CI/CD	GitHub Actions + GHCR	—	Compilación, pruebas y despliegue continuo
Dominio	CloudFlare	—	Encargado del manejo del domino del sistema.
Infraestructura	AWS EC2	—	Instancia de producción

1.2 Patrón arquitectónico

El backend es un monolito en capas organizado por dominios funcionales, (sucursales, socios y planes, asistencias, caja y facturación, recordatorios), siguiendo el patrón repoistory. Se despliega como una única aplicación Spring Boot dentro de un contenedor Docker. Se eligió este enfoque

porque el sistema tiene un solo equipo y un dominio acotado, lo que simplifica el despliegue, las transacciones y las pruebas frente a una arquitectura de microservicios.

Capa	Responsabilidad
Seguridad	Filtro JWT, autenticación, autorización por rol y política CORS.
Controller	Expone los endpoints REST, valida los DTOs de entrada y devuelve las respuestas HTTP.
Service	Contiene las reglas de negocio: vigencia de membresías, estados de factura, trazabilidad de sucursal y usuario registrador.
Repository	Acceso a datos con Spring Data JPA sobre MariaDB.
Scheduler	Procesos programados: detección de vencimientos y despacho de correos.
Persistencia	MariaDB 11 con esquema versionado por Flyway.

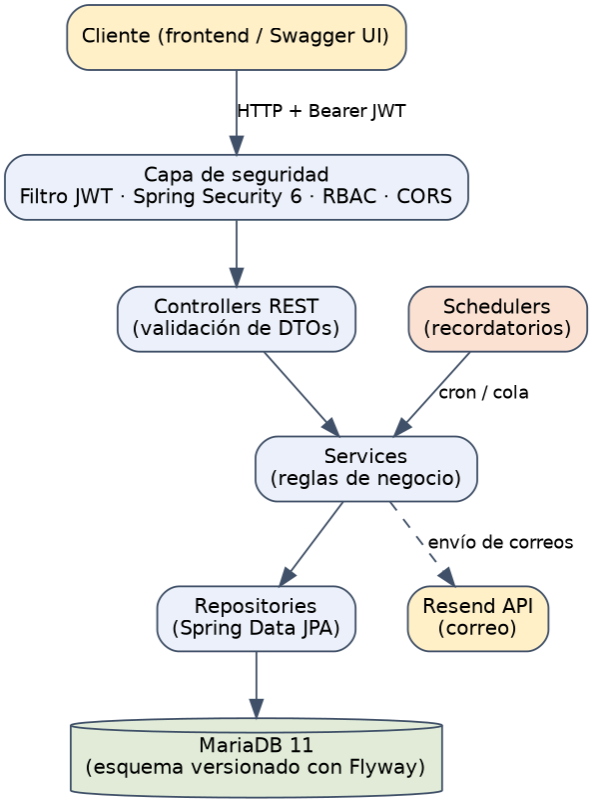


Figura 1. Arquitectura en capas del backend

1.3 Flujo general de una petición

1. El cliente envía la petición HTTP, con el encabezado Authorization: Bearer <token> en los endpoints protegidos.

2. Spring Security valida el token JWT y comprueba el rol del usuario frente al recurso solicitado.
3. El controlador valida el cuerpo con Spring Validation y delega la operación al servicio.
4. El servicio aplica las reglas de negocio y usa los repositorios JPA dentro de una transacción.
5. La respuesta se devuelve en JSON o, en el caso de comprobantes, como archivo PDF. Ante un fallo, el manejador global devuelve el formato de error estándar (sección 6.2).

2. Entorno local y setup (Developer Onboarding)

2.1 Requisitos previos

- **Git:** para clonar el repositorio.
- **Docker y Docker Compose:** versión 20 o superior, con Compose v2 recomendado. Es el único requisito de la ruta recomendada.
- **Para desarrollo sin Docker:** JDK 21 en el PATH, MariaDB 11 en el puerto 3306 y Maven 3.9+ (o el wrapper incluido ./mvnw).
- **Cliente de base de datos:** CLI de MariaDB (mariadb o mysql), DBeaver o similar, para inspeccionar los datos.

Verificación rápida de las herramientas instaladas:

```
docker --version
docker compose version
java -version      # debe indicar 21 (solo ejecución sin Docker)
mariadb --version  # opcional
```

2.2 Variables de entorno (.env)

Antes de iniciar la aplicación se crea el archivo .env en la raíz del proyecto a partir de la plantilla:

```
cp .env.example .env
```

Variable	Descripción	Valor de ejemplo / predeterminado
DB_URL	URL JDBC de conexión a MariaDB	jdbc:mariadb://db:3306/gymdb (Docker) o jdbc:mariadb://localhost:3306/gymdb (local)
DB_NAME	Nombre de la base de datos	gymdb
DB_USER	Usuario con permisos sobre la base de datos	gymuser
DB_PASSWORD	Contraseña del usuario de base de datos	gympassword123

Variable	Descripción	Valor de ejemplo / predeterminado
USER_SECURITY	Usuario de contingencia temporal en memoria	admin
PASSWORD_SECURITY	Contraseña del usuario de contingencia	admin123
JWT_SECRET	Clave secreta para firmar los JWT (mínimo 256 bits)	Cadena aleatoria segura
JWT_EXPIRATION	Vida del token JWT en milisegundos	86400000 (24 horas)
RESEND_API_KEY	API Key del servicio de correo Resend (tercero)	re_xxxxxxxxxxxxx
RESEND_FROM	Remitente de los correos de notificación	Gimnasio <onboarding@resend.dev>
DATA_INITIALIZER_ENABLED	Ejecuta el seeder inicial si la tabla de usuarios está vacía	true o false
REMINDER_ENABLED	Activa el cron de recordatorios de vencimiento	false (dev) / true (prod)
REMINDER_DAYS	Días de anticipación para avisar el vencimiento	7
REMINDER_CRON	Expresión cron del scheduler	0 0 9 * * * (todos los días a las 9 AM)
REMINDER_ZONE	Zona horaria para evaluar el cron	America/Guatemala
REMINDER_MAX_ATTEMPTS	Intentos máximos de envío de un correo	3
REMINDER_RETRY_MINUTES	Minutos de espera entre reintentos fallidos	60
REMINDER_DISPATCHER_DELAY_MS	Intervalo del despachador de cola (ms)	60000
SWAGGER_SERVER_URL	URL pública base de Swagger en producción (opcional)	https://api.tudominio.com o vacío (ruta relativa /)

Los orígenes permitidos por CORS se definen en la configuración de seguridad (sección 5.4) y no requieren variable de entorno. El archivo .env.example completo se incluye en el Anexo A.

2.3 Guía de levantamiento local

Opción A: con Docker Compose (recomendada)

1. Clonar el repositorio y entrar a la carpeta del proyecto.

```
git clone https://github.com/117CarlosCoder/Backend_Gym_Teo1.git
cd Backend_Gym_Teo1
```

2. Crear el archivo .env (sección 2.2). Para desarrollo local usar DATA_INITIALIZER_ENABLED=true y REMINDER_ENABLED=false.
3. Levantar la base de datos y el backend (entorno completo):

```
docker compose -f compose.yaml up --build -d
```

4. Alternativa para desarrollo, con live-reload y puerto de debug 5005:

```
docker compose -f compose.dev.yaml up --build
```

5. En Windows, si se requiere configuración específica de línea de comandos:

```
docker compose -f compose.win.yaml up --build
```

6. Revisar que los contenedores estén activos y ver los logs de arranque (Flyway aplica las migraciones V1 a V8):

```
docker compose ps
docker compose logs -f
```

7. Abrir Swagger UI en http://localhost:8080/swagger-ui/index.html para confirmar que la API responde.
8. Detener los contenedores:

```
docker compose down        # conserva los datos
docker compose down -v     # elimina también el volumen (base limpia)
```

Opción B: sin Docker (ejecución nativa con Maven)

1. Crear la base de datos y el usuario en MariaDB (con los valores del .env):

```
CREATE DATABASE gymdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'gymuser'@'%' IDENTIFIED BY 'gympassword123';
GRANT ALL PRIVILEGES ON gymdb.* TO 'gymuser'@'%';
FLUSH PRIVILEGES;
```

2. Cargar las variables del .env en la sesión de terminal. En local, DB_URL debe apuntar a localhost:3306:

```
set -a; source .env; set +a
export DB_URL=jdbc:mariadb://localhost:3306/gymdb
```

3. Compilar y ejecutar las pruebas:

```
./mvnw clean test
```

4. Iniciar la aplicación. Al arrancar, Flyway aplica las migraciones y el seeder carga los datos de prueba si DATA_INITIALIZER_ENABLED=true:

```
./mvnw spring-boot:run
```

La API queda disponible en http://localhost:8080.

2.4 Comandos clave

Acción	Comando
Instalación de dependencias	./mvnw clean install -DskipTests
Desarrollo	./mvnw spring-boot:run
Pruebas	./mvnw clean test
Cobertura	./mvnw clean verify

Acción	Comando
Empaquetado (JAR)	./mvnw clean package
Migraciones	Automáticas al iniciar (Flyway)
Seed de datos	DATA_INITIALIZER_ENABLED=true y arrancar

3. Base de datos y persistencia

3.1 Diagrama Entidad-Relación (DER)

La Figura 2 muestra el modelo de entidades del sistema y sus relaciones. El diagrama se puede encontrar en: https://dbdiagram.io/d/Gimnasio-DB-6a742cc0829f06bdc895f8b6.

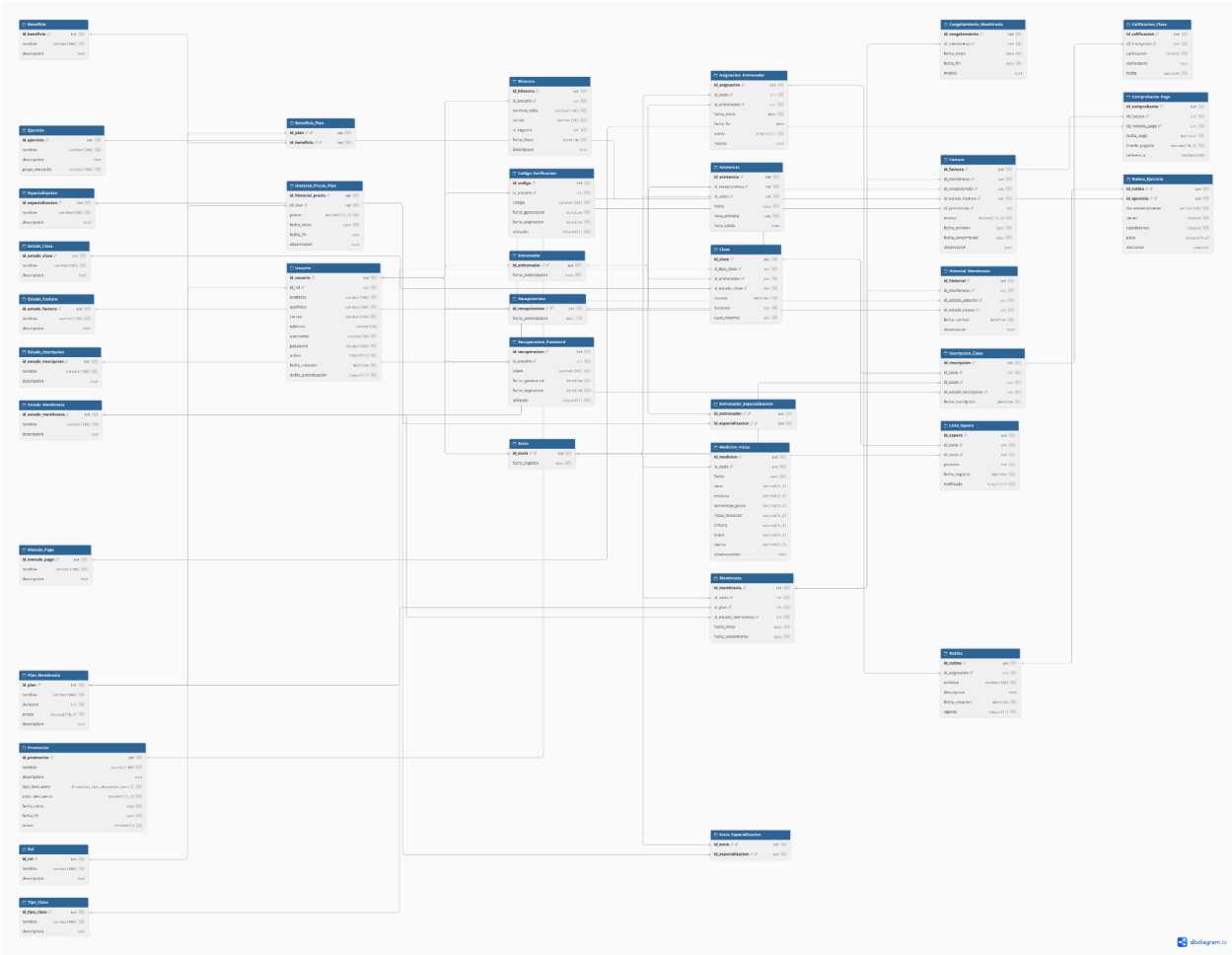


Figura 2. Diagrama Entidad-Relación

Entidad	Descripción
usuario	Cuenta de acceso al sistema con nombres, apellidos, correo único, nombre de usuario, contraseña, rol y configuración de doble autenticación.
rol	Catálogo de roles que determinan los permisos de cada usuario dentro del sistema.
socio	Registro que identifica a un socio del gimnasio y almacena su fecha de registro. Se relaciona con una cuenta de usuario.
entrenador	Registro del personal de entrenamiento, asociado a un usuario y con su fecha de contratación.
recepcionista	Registro del personal de recepción, asociado a un usuario y con su fecha de contratación.
plan_membresia	Catálogo de planes del gimnasio con nombre, duración, precio y descripción.
membresia	Contratación de un plan por parte de un socio, con estado, fecha de inicio y fecha de vencimiento.
estado_membresia	Catálogo de estados posibles de una membresía, como activa, vencida, congelada o cancelada.
historial_membresia	Registra los cambios de estado de las membresías, indicando el estado anterior, el nuevo y la fecha del cambio.

historial_precio_plan	Conserva el historial de precios de los planes, con fechas de vigencia y observaciones.
congelamiento_membresia	Registra los períodos en que una membresía queda suspendida temporalmente, incluyendo fechas y motivo.
beneficio	Catálogo de beneficios disponibles para los socios, con nombre y descripción.
beneficio_plan	Relaciona los planes de membresía con sus beneficios. Permite asignar varios beneficios a un plan y compartir beneficios entre planes.
promocion	Almacena las promociones y descuentos disponibles, incluyendo tipo de descuento, valor, período de vigencia y estado de activación.
factura	Registra los cobros asociados a una membresía, indicando el monto, las fechas de emisión y vencimiento, el estado y la promoción aplicada, si existe.
estado_factura	Catálogo de estados de las facturas, como pendiente, pagada o vencida.
comprobante_pago	Registra los pagos realizados para una factura, incluyendo monto, fecha, método de pago y referencia de la transacción.
metodo_pago	Catálogo de métodos de pago aceptados por el gimnasio, con nombre y descripción.
asistencia	Registra la entrada y salida de los socios del gimnasio, junto con la fecha y el recepcionista que registró la asistencia.

asignacion_entrenador	Vincula a un socio con un entrenador durante un período determinado, indicando fechas, estado de la asignación y motivo.
rutina	Almacena las rutinas de entrenamiento asociadas a una asignación entre un socio y un entrenador, con nombre, descripción y vigencia.
ejercicio	Catálogo de ejercicios disponibles, con nombre, descripción y grupo muscular trabajado.
rutina_ejercicio	Relaciona los ejercicios con las rutinas e indica el día de entrenamiento, las series, las repeticiones, el peso y el descanso.
especializacion	Catálogo de especializaciones relacionadas con las áreas de entrenamiento.
entrenador_especializacion	Relaciona a los entrenadores con sus especializaciones, permitiendo que un entrenador tenga varias.
socio_especializacion	Relaciona a los socios con especializaciones registradas en el sistema.
tipo_clase	Catálogo de tipos de clases que ofrece el gimnasio, con nombre y descripción.
estado_clase	Catálogo de estados de las clases, como programada, finalizada o cancelada.
clase	Registra las clases impartidas por un entrenador, con tipo, estado, horario, duración y cupo máximo.

inscripcion_clase	Registra la inscripción de un socio a una clase, indicando la fecha de inscripción y el estado correspondiente.
estado_inscripcion	Catálogo de estados de las inscripciones a clases, como confirmada, cancelada o completada.
lista_espera	Registra a los socios que esperan un lugar en una clase, indicando su posición, fecha de registro y si fueron notificados.
calificacion_clase	Almacena la calificación de una inscripción a una clase, en una escala de 1 a 5, junto con un comentario opcional.
medicion_fisica	Registra las mediciones físicas de los socios, como peso, estatura, porcentaje de grasa, masa muscular y medidas corporales.
codigo_verificacion	Almacena códigos de verificación vinculados a usuarios, con fecha de generación, vencimiento y estado de utilización.
recuperacion_password	Gestiona las solicitudes de recuperación de contraseña mediante tokens únicos con fecha de generación, expiración y estado de utilización.
bitacora	Registra las acciones realizadas por los usuarios sobre las tablas del sistema, incluyendo el tipo de acción, el registro afectado, la fecha y una descripción.

Los nombres de tablas y columnas son los de referencia del modelo; el esquema físico definitivo es el que crean las migraciones Flyway V1 a V8 en src/main/resources/db/migration.

3.2 Estrategia de migraciones

El esquema se gestiona con Flyway. Las migraciones están versionadas de V1 a V8 y se ejecutan automáticamente cada vez que la aplicación inicia, tanto con Docker como con Maven. Flyway registra lo aplicado en la tabla flyway_schema_history y solo ejecuta las migraciones pendientes.

Ejecutar las migraciones existentes

No requiere un comando aparte. Basta con arrancar la aplicación:

```
docker compose -f compose.yaml up --build -d
# o
./mvnw spring-boot:run
```

Para verificar el estado, consultar el historial:

```
SELECT installed_rank, version, description, success
FROM flyway_schema_history
ORDER BY installed_rank;
```

Crear una nueva migración

1. Crear un archivo SQL con el siguiente número de versión, con el patrón V<n>__<descripcion>.sql (dos guiones bajos). Siguiente versión disponible: V9.

```
src/main/resources/db/migration/V9__agregar_columna_notas_a_membresia.sql
```

2. Escribir solo SQL incremental:

```
ALTER TABLE membresia ADD COLUMN notas VARCHAR(255) NULL;
```

3. Nunca modificar una migración ya aplicada: Flyway valida su checksum y la aplicación no arrancará. Los cambios se hacen siempre con una migración nueva.
4. Reiniciar la aplicación y comprobar que la nueva versión aparece en flyway_schema_history con success = 1.
5. Actualizar la entidad JPA correspondiente y el DER en docs/der.png.

3.3 Datos de prueba (seeds)

Los usuarios de prueba vienen preconfigurados en las migraciones. Además, un inicializador de datos carga el seeder inicial cuando la tabla de usuarios está vacía y DATA_INITIALIZER_ENABLED=true. Para poblar una base nueva con datos de desarrollo:

1. Establecer DATA_INITIALIZER_ENABLED=true en el .env.
2. Partir de una base vacía: docker compose down -v elimina el volumen anterior.
3. Arrancar la aplicación con docker compose up o ./mvnw spring-boot:run; los datos se cargan durante el inicio.
4. Iniciar sesión en POST /auth/login con uno de los usuarios de la tabla siguiente.

Correo electrónico	Rol	Contraseña	Acceso y permisos
alejandro.garcia@gymdemo.com	ENTRENADOR	Coach123*	Gestión de clases y asistencias deportivas

Correo electrónico	Rol	Contraseña	Acceso y permisos
maria.castro@gymdemo.com	RECEPCIONISTA	Recep123*	Check-in en sucursal, registro de socios y pagos
lucia.hernandez@gymdemo.com	CLIENTE	Client123*	Consulta de perfil, membresía y notificaciones
diego.pineda@gymdemo.com	CLIENTE	Client123*	Estado de cuenta y asistencias
valeria.mendez@gymdemo.com	CLIENTE	Client123*	Consulta de membresía activa

El acceso administrativo inicial se obtiene con el usuario de contingencia definido por USER_SECURITY y PASSWORD_SECURITY. Estas credenciales son solo para desarrollo y pruebas: deben cambiarse antes de exponer cualquier entorno real.

4. Estructura del proyecto

El proyecto sigue la estructura estándar de Maven para aplicaciones Spring Boot, con los paquetes de código organizados por capa.

```
clauda-lovers-backend/
├── .github/workflows/       # Pipelines CI/CD
├── docs/                    # Documentación técnica y DER
├── src/
│   ├── main/
│   │   ├── java/.../
│   │   │   ├── config/      # Seguridad, CORS, OpenAPI, propiedades
│   │   │   ├── controller/  # Endpoints REST
│   │   │   ├── service/     # Reglas de negocio
│   │   │   ├── repository/  # Spring Data JPA
│   │   │   ├── entity/      # Entidades JPA
│   │   │   ├── dto/         # Request / Response
│   │   │   ├── security/    # Filtro JWT, UserDetails
│   │   │   ├── exception/   # Excepciones y handler global
│   │   │   ├── scheduler/   # Recordatorios y despachador
│   │   │   └── seed/        # Data initializer
│   │   └── resources/
│   │       ├── application.yml
│   │       └── db/migration/ # Flyway V1 a V8
│   └── test/java/.../       # Pruebas unitarias
├── compose.yaml             # Producción (App + MariaDB)
├── compose.dev.yaml         # Desarrollo (debug 5005)
├── compose.win.yaml         # Variante Windows
├── Dockerfile               # Imagen de producción
├── .env.example             # Plantilla de variables
├── mvnw / mvnw.cmd          # Maven Wrapper
└── pom.xml                  # Dependencias y build
```

4.1 Responsabilidad de cada carpeta

Ruta	Responsabilidad
.github/workflows/	Pipelines de GitHub Actions: CI en cada push o PR a main y CD al fusionar.

Ruta	Responsabilidad
docs/	Documentación técnica (este manual) y diagrama Entidad-Relación.
config/	Configuración global: seguridad, CORS, OpenAPI y propiedades de recordatorios.
controller/	Controllers REST que reciben las peticiones y devuelven respuestas HTTP.
service/	Reglas de negocio y transacciones.
repository/	Interfaces Spring Data JPA para el acceso a MariaDB.
entity/	Entidades JPA mapeadas a las tablas.
dto/	Objetos de transferencia (request y response) con validaciones.
security/	Filtro JWT, servicio de usuarios y utilidades de token.
exception/	Excepciones de negocio y manejador global de errores.
scheduler/	Tareas programadas de recordatorios y despachador de correos.
seed/	Inicializador de datos de desarrollo.
resources/db/migration	Scripts SQL de Flyway (V1 a V8).
src/test/java	Pruebas unitarias ejecutadas con ./mvnw clean test.
compose*.yaml	Orquestación Docker: producción, desarrollo con debug y variante Windows.
Dockerfile	Imagen optimizada de producción.
.env.example	Plantilla de variables de entorno.

5. Seguridad y autenticación

5.1 Flujo de autenticación

La autenticación es sin estado (no se almacena ni se mantiene información de sesión), basada en JSON Web Tokens (JWT) con Spring Security 6 y la librería JWT.

1. El cliente envía sus credenciales a POST /auth/login.
2. El servidor las valida y responde con un token JWT firmado con JWT_SECRET.
3. En cada petición posterior el cliente envía el token en el encabezado Authorization: Bearer <token>.
4. El filtro JWT verifica la firma y la vigencia, extrae el rol y establece el contexto de seguridad.
5. Si el token falta, es inválido o expiró, el servidor responde 401. Si el rol no tiene permiso sobre el recurso, responde 403.

```
POST /auth/login
Content-Type: application/json

{
 "email": "maria.castro@gymdemo.com",
 "password": "Recep123*"
}

Respuesta 200:
{ "token": "eyJhbGciOiJIUzI1NiJ9..." }
```

5.2 Gestión de sesión

Aspecto	Detalle
Duración del token	86400000 ms (24 horas), configurable con JWT_EXPIRATION
Refresh token	No se implementa. Al expirar el token, el usuario inicia sesión de nuevo en /auth/login
Almacenamiento	El servidor no guarda sesión (stateless). El token lo conserva el cliente; en Swagger se pega en el botón Authorize
Firma	HMAC con JWT_SECRET, de mínimo 256 bits
Revocación	No hay lista de revocación; la expiración del token limita su vida útil. Rotar JWT_SECRET invalida todos los tokens emitidos

5.3 Matriz de permisos (RBAC)

El sistema define cuatro roles. La matriz resume qué recursos puede usar cada uno.

Recurso / capacidad	ADMIN	RECEPCIONISTA	ENTRENADOR	CLIENTE
Gestión de sucursales	Sí	No	No	No
Alta de socios y planes	Sí	Sí	No	No
Membresías: emitir, renovar, congelar, cancelar	Sí	Sí	No	No
Check-in / registro de ingresos	Sí	Sí	No	No
Pagos y facturación (emisión y PDF)	Sí	Sí	No	Solo consulta propia
Clases grupales y asistencias a clases	Sí	No	Sí	No
Perfil, membresía y notificaciones propias	Sí	Sí	Sí	Sí

Los permisos se aplican a nivel de endpoint con la configuración de rutas de Spring Security y anotaciones de método (@PreAuthorize).

5.4 Políticas de CORS y Rate Limiting

CORS

Parámetro	Valor
Orígenes permitidos	Solo el dominio del frontend de producción y http://localhost:3000, http://localhost:4200 y http://localhost:5173 en desarrollo
Métodos	GET, POST, PUT, PATCH, DELETE, OPTIONS
Cabeceras permitidas	Authorization, Content-Type, Accept
Credenciales	No se usan cookies; el token viaja en la cabecera Authorization
Swagger en producción	Usa rutas relativas y cabeceras X-Forwarded-*, por lo que no genera conflictos de CORS

Rate Limiting

El límite de peticiones se aplica con CloudFlare, antes de que la petición llegue a la aplicación:

Zona	Límite	Ámbito
Global de la API	60 peticiones por minuto por IP (ráfaga de 20)	Todos los endpoints
Autenticación	10 peticiones por minuto por IP	POST /auth/login

Al superar el límite, CloudFlare mostrará una advertencia de rate limiting.

5.5 Buenas prácticas de seguridad

- Generar JWT_SECRET con al menos 256 bits aleatorios (por ejemplo, openssl rand -base64 48) y no reutilizarlo entre entornos.
- Cambiar las credenciales del usuario de contingencia y de los seeders fuera de desarrollo.
- Guardar RESEND_API_KEY, DB_PASSWORD y JWT_SECRET como secretos de GitHub Actions y variables del servidor; nunca en el repositorio.
- Desactivar DATA_INITIALIZER_ENABLED en producción una vez cargados los datos reales.
- Servir la API por HTTPS en producción.

6. Swagger, excepciones y tareas en segundo plano

6.1 Swagger UI

Entorno	URL
Local	http://localhost:8080/swagger-ui/index.html
Producción (EC2)	http://<IP_O_DOMINIO_EC2>:8080/swagger-ui/index.html (o https://<DOMINIO>/swagger-ui/index.html)
Especificación OpenAPI (JSON)	http://<IP_O_DOMINIO>:8080/v3/api-docs

Cómo autenticarse en Swagger:

1. Hacer login en POST /auth/login con uno de los usuarios de prueba.
2. Copiar el token JWT de la respuesta.
3. Pulsar el botón Authorize en la parte superior de Swagger UI.
4. Pegar únicamente el token, sin la palabra Bearer, y confirmar.

6.2 Formato estándar de respuesta de error

Todos los errores se devuelven en JSON con la misma estructura, generada por un manejador global (@RestControllerAdvice):

```
{
 "statusCode": 400,
 "message": "El correo es obligatorio",
 "error": "Bad Request",
 "timestamp": "2026-10-28T15:04:05Z"
}
```

Campo	Descripción
statusCode	Código HTTP de la respuesta
message	Descripción legible del error; en validaciones, el detalle de los campos inválidos
error	Nombre estándar del estado HTTP
timestamp	Fecha y hora del error en formato ISO 8601

Código	Cuándo se devuelve
400 Bad Request	Datos inválidos o reglas de negocio incumplidas (por ejemplo, membresía vencida)
401 Unauthorized	Token ausente, inválido o expirado; credenciales incorrectas
403 Forbidden	El rol del usuario no tiene permiso sobre el recurso
404 Not Found	El recurso solicitado no existe
409 Conflict	Conflicto de estado o duplicado (por ejemplo, correo ya registrado)
429 Too Many Requests	Se superó el límite de peticiones
500 Internal Server Error	Error no controlado; el detalle se registra en los logs, no en la respuesta

6.3 Cron jobs y colas de trabajo

El proceso asíncrono del sistema es el de recordatorios automatizados de vencimiento de membresías. Un scheduler revisa las fechas de vencimiento y encola correos que se envían

mediante la API de Resend. No se usa un broker externo: la cola la gestiona la propia aplicación con un despachador periódico.

Parámetro	Variable	Valor por defecto
Activación	REMINDER_ENABLED	false en desarrollo, true en producción
Horario del cron	REMINDER_CRON	0 0 9 * * * (todos los días a las 9:00)
Zona horaria	REMINDER_ZONE	America/Guatemala
Anticipación del aviso	REMINDER_DAYS	7 días antes del vencimiento
Intentos máximos	REMINDER_MAX_ATTEMPTS	3
Espera entre reintentos	REMINDER_RETRY_MINUTES	60 minutos
Intervalo del despachador	REMINDER_DISPATCHER_DELAY_MS	60000 ms (1 minuto)

Funcionamiento paso a paso:

1. A las 9:00 (hora de Guatemala) el scheduler detecta las membresías que vencen dentro de REMINDER_DAYS días y registra un recordatorio pendiente por cada una.
2. Cada minuto, el despachador toma de la cola los recordatorios pendientes y los envía por correo con Resend.
3. Si el envío falla, el recordatorio se reprograma 60 minutos después y se incrementa su contador de intentos.
4. Al llegar a 3 intentos fallidos, el recordatorio se marca como fallido y no se reintenta más.

Para probarlo en desarrollo: poner REMINDER_ENABLED=true y un REMINDER_CRON frecuente (por ejemplo 0 * * * * * para cada minuto), con una RESEND_API_KEY de pruebas.

7. Estrategia de tests y CI/CD

7.1 Pruebas

Tipo	Comando	Descripción
Pruebas unitarias	./mvnw clean test	Prueban servicios y reglas de negocio de forma aislada; son las que corre el CI
Pruebas de integración	./mvnw verify	Prueban controllers, seguridad y repositorios contra una base MariaDB 11 (en CI, un contenedor del runner)
Una clase concreta	./mvnw test -Dtest=NombreDeLaClaseTest	Ejecuta solo esa clase

7.2 Pipeline de despliegue

El proyecto usa GitHub Actions con dos etapas. Los entornos son: local (Docker Compose o Maven), CI (runner de GitHub) y producción (AWS EC2).

Integración continua (CI)

- Se dispara con cada push o pull request a la rama main y develop.
- El runner de GitHub Actions levanta un contenedor MariaDB 11, compila el proyecto y ejecuta las pruebas.
- Si alguna prueba falla, el PR no se puede fusionar.

Entrega continua (CD)

1. Se activa al fusionar cambios aprobados en main.
2. Se construye la imagen Docker optimizada de producción.
3. Se publica en GitHub Container Registry (GHCR) con un tag versionado y :latest.
4. Por SSH seguro, el workflow actualiza la instancia AWS EC2: reemplaza el contenedor del backend sin afectar la persistencia de datos y reinicia el proxy inverso Nginx.
5. Flyway aplica automáticamente las migraciones nuevas cuando el contenedor actualizado arranca.

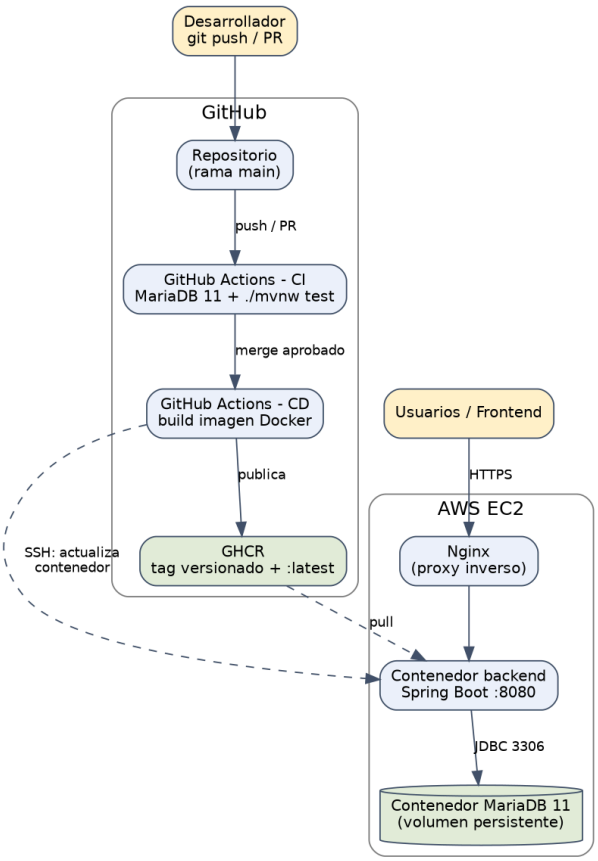


Figura 3. Flujo de CI/CD y despliegue en AWS EC2

Secretos del pipeline

Los secretos se configuran en GitHub (Settings > Secrets and variables > Actions) y en el servidor: credenciales SSH de la instancia EC2, DB_PASSWORD, JWT_SECRET, RESEND_API_KEY y las demás variables de la sección 2.2.

Reversión

Como cada imagen se publica con un tag versionado, una versión defectuosa se revierte desplegando en EC2 el tag anterior; la base de datos se conserva en su volumen.

8. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
La app no conecta a la base de datos	DB_URL apunta a db (Docker) pero se ejecuta con Maven	Usar jdbc:mariadb://localhost:3306/gymdb en ejecución nativa
Error de checksum de Flyway al arrancar	Se modificó una migración ya aplicada	Restaurar el archivo original y crear una migración nueva

Síntoma	Causa probable	Solución
401 en todos los endpoints	Token expirado o pegado con la palabra Bearer en Swagger	Hacer login de nuevo y pegar solo el token
No existen usuarios de prueba	DATA_INITIALIZER_ENABLED=false o la tabla ya tenía datos	Activar la variable y reiniciar con base limpia (docker compose down -v)
Puerto 8080 o 3306 ocupado	Otro proceso usa el puerto	Detener ese proceso o cambiar el puerto expuesto en el compose
No llegan los correos	RESEND_API_KEY inválida o REMINDER_ENABLED=false	Verificar la clave y activar los recordatorios
Error de versión de Java	JDK distinto de 21	Instalar JDK 21 y revisar java -version

Anexo A. Archivo .env.example

Contenido del archivo .env.example, con todas las variables que usa el sistema. Los valores son de ejemplo y deben reemplazarse en cada entorno.

```
# ===== Base de datos =====
DB_URL=jdbc:mariadb://db:3306/gymdb
DB_NAME=gymdb
DB_USER=gymuser
DB_PASSWORD=gympassword123

# ===== Seguridad =====
USER_SECURITY=admin
PASSWORD_SECURITY=admin123
JWT_SECRET=cambia_esta_clave_por_una_cadena_aleatoria_de_minimo_256_bits
JWT_EXPIRATION=86400000

# ===== Correo (Resend) =====
RESEND_API_KEY=re_xxxxxxxxxxxxx
RESEND_FROM=Gimnasio <onboarding@resend.dev>

# ===== Seeder =====
DATA_INITIALIZER_ENABLED=true

# ===== Recordatorios de vencimiento =====
REMINDER_ENABLED=false
REMINDER_DAYS=7
REMINDER_CRON=0 0 9 * * *
REMINDER_ZONE=America/Guatemala
REMINDER_MAX_ATTEMPTS=3
REMINDER_RETRY_MINUTES=60
REMINDER_DISPATCHER_DELAY_MS=60000

# ===== Swagger (opcional) =====
SWAGGER_SERVER_URL=
```

Anexo B. Glosario

Término	Significado
JWT	JSON Web Token: credencial firmada que identifica al usuario en cada petición.
RBAC	Control de acceso basado en roles.
DTO	Objeto de transferencia de datos entre cliente y servidor.
JPA	Especificación de Java para el mapeo objeto-relacional.
Flyway	Herramienta de migraciones versionadas de base de datos.
GHCR	GitHub Container Registry, donde se publican las imágenes Docker.
CI/CD	Integración continua y entrega continua.
Seeder	Proceso que carga datos iniciales de prueba.
Cron	Expresión que define cuándo se ejecuta una tarea programada.